

512 × 512 INVERTED HASH

A Novel Cryptographic Hash Function Based on Pyramidal Reduction with Sacred Pair Inversion

Inversion at Dimension 257 — K1 ↔ K2 — 2048-Bit Output Space

Marco Lazcano

Independent Researcher in Cryptography

969 Clubhouse Circle W, Apt F — Decatur, GA 30032, USA

roble99oak77@gmail.com

May 2026

Input 256 bytes	Output 256 bytes	State 2048 bits	Rounds 24
---------------------------	----------------------------	---------------------------	---------------------

Abstract

We present the 512×512 Inverted Hash, a novel cryptographic hash function based on a pyramidal reduction architecture with a structural inversion mechanism. The function introduces the concept of Sacred Pair inversion at dimension 257 — a mathematical boundary between complementary spaces K1 and K2, where each byte x maps to its bitwise complement ($\sim x$ & 0xFF). The design operates on 256-byte input blocks, maintains a 256-byte (2048-bit) internal state, and produces a 256-byte output through 24 permutation rounds. Each round processes the full 256-byte state as a single pyramidal block, achieving complete diffusion across all 2048 output bits. Eleven independent statistical tests on the complete 2048-bit output demonstrate: avalanche effect of 49.906% with the remarkably tight standard deviation of 1.104%; Shannon entropy of 7.9974 bits/byte (99.97% efficiency); Chi-square of 237.51; Pearson correlation average of 0.02524 over 32,640 byte pairs at N=1,000 samples; Strict Avalanche Criterion deviation of 1.09%; and zero collisions in 400 attempts. This function is a research prototype; formal cryptographic audit is required before any production use.

Keywords: *hash function, cryptography, pyramidal reduction, sacred pair inversion, sponge construction, 2048-bit output, K1/K2 boundary, AES S-box.*

1. Introduction

Cryptographic hash functions are foundational primitives of modern information security. The design space for hash functions has been dominated by a small number of paradigms: Merkle-Damgård

constructions (MD5, SHA-1, SHA-2) and the sponge construction (SHA-3/Keccak). Each paradigm encodes specific assumptions about how input entropy should be absorbed and diffused across the output space.

This work explores a structurally distinct approach: pyramidal reduction. Instead of processing the state in fixed-width word operations, the function iteratively compresses the state through a pyramid of transformations, where each level produces a shorter sequence from the previous one until a single byte — the pyramid tip — is reached. The tip and the sequence of left anchors from each level constitute the output of the compression function.

The central novelty of this work is the Sacred Pair inversion: a structural boundary at dimension 257 that maps K1 to K2 via bitwise complementation. This boundary is a reserved property — with standard 256-byte inputs, the pyramid produces 255 anchors, placing all active processing within K1 (dimensions 0–254). The K2 region (dimension 257 and beyond) is defined as a structural extension for future work, providing a mathematical framework for bidirectional hash architectures.

This paper makes the following contributions:

- Formal definition of the pyramidal nonlinear compression function with full 256-byte block processing.
- Sacred Pair inversion mechanism defining the K1/K2 dimensional boundary at position 257.
- Complete sponge-type architecture with 2048-bit state, standard padding, and 24-round permutation.
- Rigorous empirical evaluation through 11 independent statistical tests on the full 2048-bit output.
- Transparent documentation of design evolution including the two architectural fixes (Fix A and Fix B) that resolved an initial diffusion limitation.

2. Background and Theoretical Framework

2.1 Properties of a Secure Hash Function

A cryptographic hash function $H: \{0,1\}^* \rightarrow \{0,1\}^n$ must satisfy:

- Preimage resistance: given h , infeasible to find m such that $H(m) = h$.
- Second preimage resistance: given m , infeasible to find $m' \neq m$ such that $H(m') = H(m)$.
- Collision resistance: infeasible to find any pair (m, m') with $H(m) = H(m')$.

2.2 Sponge Construction

The sponge construction (Bertoni et al., SHA-3) absorbs data into a rate portion and protects security through a hidden capacity. The 512×512 Inverted Hash follows this paradigm: 256-byte absorption blocks permute a 256-byte state, with the output squeezed from successive permutations.

2.3 AES S-box

The AES S-box provides non-linearity (NL=112) and a maximum differential probability of 1.56%, well below the 6.25% security threshold. It is used unchanged throughout the permutation, leveraging its extensively validated cryptographic properties.

2.4 Sacred Pair: K1 ↔ K2

The Sacred Pair operation is defined as: $SP(x) = (\sim x) \& 0xFF$, the bitwise complement of x modulo 256. This operation satisfies two fundamental properties: (1) involution — $SP(SP(x)) = x$ for all x ; and (2) maximum XOR distance — $x \text{ XOR } SP(x) = 0xFF$ for all x . These properties define a perfect pairing across the 256-element byte space, partitioning it into 128 complementary pairs. The boundary at dimension 257 represents the transition from standard pyramidal processing (K1) to the complemented inversion space (K2), providing a structural extension point for bidirectional architectures.

3. Design of the 512×512 Inverted Hash

3.1 Global Parameters

Parameter	Value and Rationale
State size	256 bytes (2048 bits) — 1024-bit birthday bound
Input block	256 bytes per absorption round
Standard output	256 bytes (2048 bits) — 4× SHA-512
Rounds	24 — Fix B: increased from 12 for complete diffusion
Chunk size	256 bytes — Fix A: full state as one pyramidal block
S-box	AES S-box (NL=112, max differential 1.56%)
Round constants	16 × 32-bit values from hexadecimal digits of π
Sacred Pair	$SP(x) = (\sim x \& 0xFF)$ — K1/K2 boundary at dim. 257

3.2 Pyramidal Nonlinear Function

The core compression function takes the full 256-byte state vector and reduces it iteratively through a pyramid of transformations. At each level, adjacent bytes are combined:

```
diff = SBOX[(seq[i+1] - seq[i]) mod 256] # AES S-box substitution
diff = safe_byte(diff XOR anchor)         # anchor mixing
diff = rotate_left(diff, (i mod 7) + 1)   # position-dependent rotation
```

The reduction continues until a single byte remains — the pyramid tip. The left anchor (first element) of each level, combined with the tip, forms the inverted base output. `safe_byte()` enforces strict 8-bit boundary safety at every step.

3.3 Sacred Pair Inversion

After collecting left anchors from all pyramid levels, the inversion function applies:

```
inverted[i] = SP(anchor) if i >= 256 else anchor
```

$$SP(x) = (\sim x) \ \& \ 0xFF$$

With 256-byte input blocks, the pyramid produces 255 anchors (indices 0–254), so the Sacred Pair branch ($i \geq 256$) defines the K1/K2 structural boundary — a reserved dimension for future bidirectional architectures. The inverted anchors are then reversed and prepended with the pyramid tip to form the 256-byte output of the compression function.

3.4 Internal Permutation — 24 Rounds (Fix A + Fix B)

The permutation applies 24 rounds to the full 256-byte state. Each round consists of three steps:

- Full-state pyramidal mixing (Fix A): the entire 256-byte state is processed as one pyramidal block — replacing the original 64-byte chunk approach that caused incomplete diffusion.
- Linear diffusion: each byte is XOR-ed with the next byte, propagating influence across the entire state.
- Nonlinear substitution: AES S-box applied to every byte of the state.

The number of rounds was increased from 12 to 24 (Fix B) to ensure complete diffusion across all 2048 bits. The combination of Fix A and Fix B transformed the avalanche effect from 30.94% (std=13.63%) to 49.906% (std=1.104%).

3.5 Padding and Absorption

Padding follows a SHA-like scheme: append 0x80, zero-pad until `buffer_length mod 256 = 248`, then append the message length in bits as a 64-bit little-endian integer. This guarantees domain separation between messages of different lengths.

4. Design Evolution: From v1 to v2

Transparent documentation of the design evolution is an integral part of the scientific integrity of this work. The initial implementation (v1) used 64-byte chunks and 12 rounds, producing the following results:

Test	v1 (chunk=64, 12r)	v2 (chunk=256, 24r)	
Avalanche mean	30.94%	49.906%	✓
Std deviation	13.63%	1.104%	✓
SAC deviation>10%	75.27%	1.09%	✓
Entropy	7.9979	7.9974	✓
Chi-square	275.30	237.51	✓

Fix A (full-state pyramidal block) resolved the root cause: with 64-byte chunks, a bit change in one chunk required 12 rounds of linear diffusion to reach the other three chunks — insufficient for complete mixing. Processing the full 256-byte state as one pyramidal block ensures every input bit influences the compression output of every round directly. Fix B (24 rounds) provides additional mixing margin and further tightens the avalanche standard deviation.

5. Experimental Results

All eleven tests were performed on the fully validated v2 implementation. Inputs generated with `secrets.token_bytes()` — cryptographically uniform. All tests operate on the complete 256-byte (2048-bit) output.

5.1 Complete Results Summary

Test	Result	Norm / Threshold	
Basic functionality	All distinct	All distinct	✓
Determinism (200 trials)	0 errors	0 errors	✓
Avalanche mean (2048 bits)	49.906%	50.00% ± 2%	✓
Std deviation	1.104%	< 5%	✓
Shannon entropy	7.9974 b/byte	> 7.98	✓
Entropy efficiency	99.97%	> 99%	✓
Chi-square	237.51	< 293.2 (95%)	✓
Pearson corr. avg (N=1000)	0.02524	< 0.05	✓
Pearson corr. max	0.13515	< 0.50	✓
S-box max differential	1.56%	< 6.25%	✓
SAC deviation > 10%	1.09%	< 5%	✓
Collision resistance	0 / 400	0	✓
Sacred pair properties	Verified	Involution + XOR	✓

5.2 Avalanche Effect — Full 2048 Bits

150 avalanche tests were performed on the complete 256-byte (2048-bit) output. A single input bit was flipped at a random position among 256 input bytes, and all 2048 output bits were compared. The mean of 49.906% with $\sigma = 1.104\%$ is the tightest standard deviation recorded across both hash designs in this research program. This indicates not only correct average behavior but exceptional consistency — every single input bit change produces near-uniform mixing across all 2048 output bits.

5.3 Shannon Entropy and Uniformity

Shannon entropy of 7.9974 bits/byte (99.97% efficiency) was measured over 76,800 bytes from 300 samples. The Chi-square statistic of 237.51 — the lowest value recorded in this research — is well below both the 95% critical value (293.2) and the 99% critical value (310.4), confirming statistically excellent output uniformity across all 256 possible byte values.

5.4 Pearson Correlation — Definitive Result at N=1,000

Pearson correlation was measured over all 32,640 byte pairs of the 256-byte output using N=1,000 samples — a sample size selected specifically to achieve statistical definitiveness. The pairs-to-samples ratio of 32.6 is sufficient to distinguish structural correlation from sampling variance. The average correlation of 0.02524 confirms genuine independence between output bytes. Only 52 of 32,640 pairs (0.16%) exceeded the 0.10 threshold, consistent with expected statistical noise at this sample size.

5.5 Strict Avalanche Criterion (SAC)

The SAC was evaluated across 8 input bit positions and all 2048 output bit positions over 150 samples. Only 179 of 16,384 cells (1.09%) showed deviation greater than 10% from the ideal 50% flip probability. This result represents a dramatic improvement over the v1 result of 75.27% — confirming that Fix A (full-state pyramidal block) resolved the structural diffusion limitation completely.

5.6 Sacred Pair Verification

The Sacred Pair operation was verified to satisfy both defining properties across all 256 byte values: (1) involution — $SP(SP(x)) = x$ for all $x \in [0, 255]$; and (2) maximum XOR distance — $x \oplus SP(x) = 0xFF$ for all x . These properties confirm the mathematical integrity of the K1/K2 boundary definition.

5.7 Sensitivity to Minimal Inputs

Input	Hash — first 32 hex characters
b"" (empty)	36ede6bdc0eeff49375d94935936cf52
b"\x00"	ecccf4678a7b4075e67a8fd4f9433954
b"\x01"	95feb8f924c64adfe1b6bc724cd741d3
b"\xff"	915679b2a4a0cfbccdcfef01eef5a297
b"Hola Mundo"	7810e0e497bd6e29a186fb12129175d9
b"Hello"	84d74494575458ff0677f0119bc26b70
b"hello"	00b9bff83f086e9b8d1d5e7d0c7a168d
b"hello " (sp.)	f5dfb7b95923739a1aa7a11472de140b

6. Performance

The current implementation is written in pure Python 3, serving for conceptual and statistical validation:

Input size	Time / hash	Note
256 bytes	~1,444 ms	Prototype only
512 bytes	~1,924 ms	Prototype only
1 KB	~2,885 ms	Prototype only

Input size	Time / hash	Note
4 KB	~8,658 ms	<i>Prototype only</i>

The performance cost relative to the Pyramidal Hash 256×512 (121 ms / 64 bytes) reflects the larger state (256 vs 64 bytes per chunk) and doubled round count. A C or Rust implementation with SIMD optimizations is expected to reduce latency by approximately 100×.

7. Discussion

7.1 Strengths

- Avalanche of 49.906% with $\sigma = 1.104\%$ — the tightest standard deviation in this research program, indicating exceptional consistency across all 2048 output bits.
- SAC of 1.09% — only 179 of 16,384 output bit positions show deviation above 10%, confirming near-ideal bit independence.
- Pearson correlation of 0.02524 at N=1,000 — statistically definitive independence across all 32,640 byte pairs.
- Chi-square of 237.51 — excellent output uniformity, the lowest value recorded in this research.
- Sacred Pair provides a mathematically well-defined structural boundary with provable involution and maximum XOR distance properties.
- Transparent design evolution: the paper documents both v1 and v2, providing a complete scientific record.

7.2 Limitations and Future Work

- No formal cryptanalysis: differential, linear, algebraic, and length-extension attacks have not been formally studied. Required before any non-academic use.
- Sacred Pair K2 activation: with standard 256-byte inputs, the K2 region (dimension 257+) is never reached. Exploring larger block sizes or explicit K2 activation is a natural extension.
- Performance: pure Python is unsuitable for production. C/Rust implementation planned.
- External audit: independent cryptographic peer review is required.

8. Conclusion

The 512×512 Inverted Hash demonstrates exceptional statistical properties across all eleven empirical tests on the complete 2048-bit output. The avalanche effect of 49.906% with $\sigma = 1.104\%$ is the most consistent result recorded in this research. The SAC result of 1.09% — transformed from 75.27% through architectural refinement — confirms complete diffusion across all 2048 output bits. The Pearson correlation of 0.02524 at N=1,000 provides statistically definitive evidence of output byte independence.

The Sacred Pair mechanism introduces a novel structural concept — the K1/K2 dimensional boundary at position 257 — that distinguishes this design from existing hash architectures. While

the K2 region is currently reserved as a structural property, it provides a mathematically precise framework for future bidirectional hash architectures.

The transparent documentation of design evolution from v1 to v2, including the identification and resolution of the diffusion limitation through Fix A and Fix B, exemplifies the scientific process and provides a complete record for peer review. Full source code is publicly available for independent reproduction and analysis.

⚠ **Security Warning:** This hash function is a research prototype. It has not been formally audited by the cryptographic community. It must not be used in production systems, security applications, or cryptographic protocols without independent formal review.

References

- [1] Daemen, J. & Rijmen, V. (2002). The Design of Rijndael: AES. Springer.
- [2] Bertoni, G., Daemen, J., Peeters, M. & Van Assche, G. (2011). Cryptographic sponge functions. IACR.
- [3] NIST (2015). SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions. FIPS PUB 202.
- [4] NIST (2012). SHA-2 Standard: Secure Hash Standard. FIPS PUB 180-4.
- [5] Menezes, A., van Oorschot, P. & Vanstone, S. (1996). Handbook of Applied Cryptography. CRC Press.
- [6] Preneel, B. (1993). Analysis and Design of Cryptographic Hash Functions. KU Leuven.
- [7] Biham, E. & Shamir, A. (1991). Differential cryptanalysis of DES-like cryptosystems. Journal of Cryptology, 4(1), 3–72.
- [8] Lazcano, M. (2026). Pyramidal Hash 256×512: Design, Analysis and Evaluation of a Novel Cryptographic Hash Function. Independent Research.